A Verification-Gated Agentic RTL→GDSII Flow

Where the Gate Cannot Lie

IEEE SSCS PICO Open-Source Chipathon 2026

Track D (AI / LLM-assisted Circuits)

Team SystemsGenesys

August 2026

Table of Contents

Submission

Team	SystemsGenesys
Competition	IEEE SSCS PICO Open-Source Chipathon 2026
Track	Track D — AI / LLM-assisted Circuits
Demonstration vehicle	PRESENT-80 block cipher on gf180mcuD
Artifact status	Built and tested (milestones M0–M24)
Date	August 2026

Team members

Name	Role
Batzolis Eleftherios (team lead)	Team Lead — agentic flow & control graph
Dr. Rantos Konstantinos	Project Overview
Dr. Drosatos Georgios	Project Overview

One-line thesis. Trust in an agentic Chat→RTL→GDSII flow should come from architecture, not from the model's self-report: no stage advances on the model's own assessment, so a hijacked or mistaken agent cannot certify its own output into silicon.

1. Thesis

Trust in an agentic RTL→GDSII flow should come from **architecture, not from the model's self-report.** In our pipeline no stage advances on the model's own assessment: every gate reads `gate_ok` from a real tool oracle, behavioural gates check the design against *ground-truth* spec vectors rather than a model-generated oracle, a green-but-vacuous result is refused rather than promoted, tool execution is isolated in a network-less sandbox, and every decision lands in a tamper-evident, HMAC-signed provenance chain. A hijacked or simply-mistaken agent therefore **cannot certify its own output into silicon.**

The contribution is a security *architecture* demonstrated by construction and by targeted defense case studies (§7).

---

2. Motivation: the dangerous failure is a lying gate

Agentic chip-design pipelines ingest specifications and reference material from untrusted sources and invoke EDA tools with broad system access. Two failure modes follow, and they collapse into the same operational question:

- **Manipulated agent** — an adversarial specification embeds instructions that steer the agent into a subtly-degraded or backdoored implementation while the spec still looks legitimate to a human reviewer.
- **Mistaken agent** — no adversary at all: the model produces wrong RTL and then *reports success*, because the check it ran was too weak, too short, or self-graded.

The shared question: **can the flow be advanced by an assertion the agent itself controls?** If yes, both the attacker and the honest-but-wrong model win. The most dangerous artifact is not a wrong answer in a chat window — it is a fabricated chip that *passed every check* because the check was the thing that got compromised.

Our 2026-08-08 PRESENT-80 run is the concrete instance: a functionally-broken cipher reported `passed=1/1, gate_ok=true` because the differential stimulus window was too short to ever reach `done` (see `docs/KNOWN_ISSUES.md`; memory `proj_diff_tb_window_too_short`, `proj_present80_rtl_bugs`). No adversary was involved. The gate lied because the model graded itself against a weak oracle. That single observation is the spine of this design.

---

3. Architecture in brief

A verification-gated state machine over the LibreLane spine, seven stages:

`SPEC → PLAN → RTL → SYNTH → PHYSICAL → SIGNOFF → (human gate) → GDSII`

Each stage advances only when its gate reads `gate_ok` from real tool output. Each stage handler runs a **two-loop repair** — an inner syntactic loop (lint + compile-elaborate) and an outer semantic loop (simulation → typed `FailureDiagnosis`) — with a bounded escalation ladder `INNER → OUTER → EXHAUSTED → HUMAN`. State between stages is carried as typed, content-addressed Pydantic artifacts in an append-only store, never as freeform text (`docs/ARCHITECTURE.md`, `CLAUDE.md`).

Security is **not** a property of the orchestrator. It is enforced at independent layers *below* the LangGraph control plane, so a compromise at the orchestration layer does not cascade into gate authority, tool access, or network egress. §4 walks those layers with their implementation.

Technical implementation

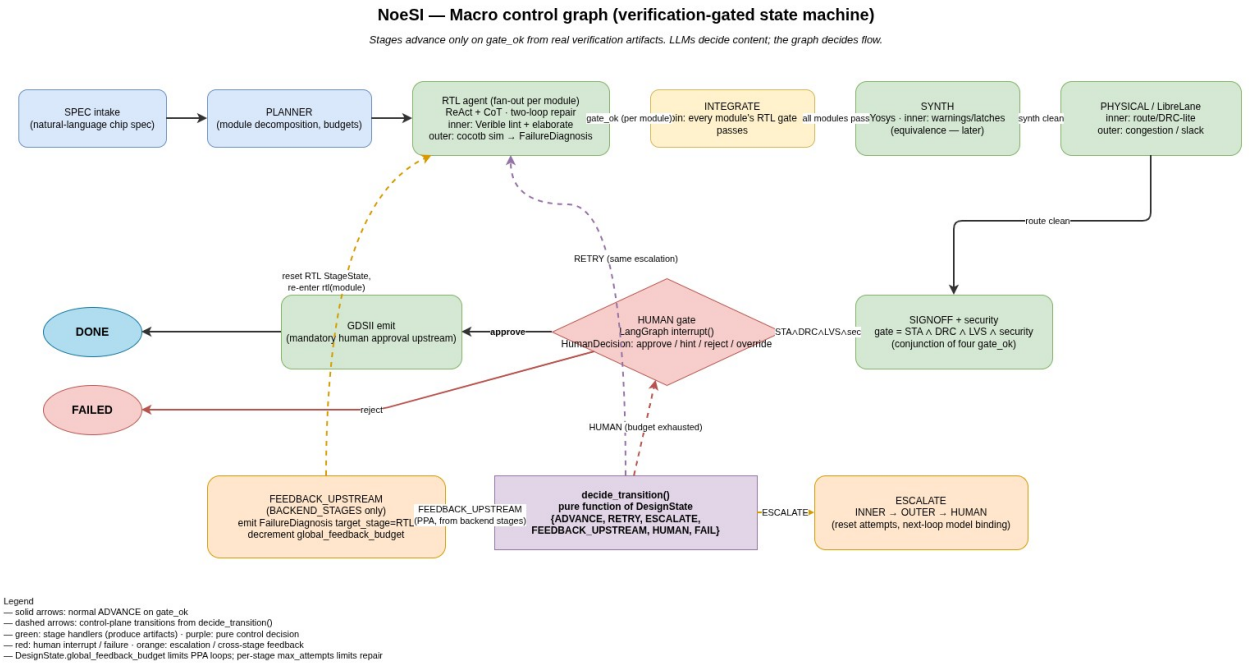
NoeSI (the `chip-agent` package) is a Python 3.12 codebase, **Pydantic v2** throughout, kept `ruff` + `mypy --strict` clean. The **control plane** is a **LangGraph** `StateGraph` whose single state object is the typed `DesignState`; checkpointing and the human-gate interrupt use `langgraph-checkpoint-sqlite`, so a run pauses and resumes across process boundaries from a SQLite checkpoint. **NoeSI works with both closed- and open-source LLMs, and mixes them by design.** Models are a routed resource behind **LiteLLM**, which gives one uniform `generate` / `stream` interface over every backend, so the same prompt and the same typed Pydantic schemas drive a closed-source frontier API and a local open-source model interchangeably.

The routing policy splits the traffic by *task* and *escalation level*: high-volume, high-temperature work — candidate RTL generation and syntactic (inner-loop) repair — routes to a **local open-source** model served by **Ollama** (Qwen-Coder class), while planning, semantic diagnosis, reflection routing, and the `EXHAUSTED`-rung repair route to a **closed-source frontier** model (e.g. Claude Opus-class) over its API. Every binding is config-overridable (`configs/*.yaml`, `docs/model_routing.md`), and a `BackendUnreachableError` transparently fails a call over to a configured fallback model.

This is also a reproducibility split: open-source runs are pinned by weights hash and are bit-for-bit replayable, while closed-source runs are replayed from frozen

invocation capture (provider / model / seed / temperature recorded in provenance). The **tool plane** wraps every EDA capability as a typed service that shells into the pinned **IIC-OSIC-TOOLS** container — **LibreLane** (driven through its Python Step/Flow API), Yosys, OpenROAD, Magic, Netgen, KLayout, Verilator, iverilog, cocotb, Verible, and OpenSTA over the gf180mcuD / sky130A PDKs - executed under `docker run --network=none` (§4.5).

The **data plane** is a content-addressed artifact store (a SQLite index plus a blob directory keyed by content hash) holding immutable typed artifacts; the **observability plane** is the HMAC-signed hash-chained audit log (§4.8) plus a JSONL / Langfuse trace. The operator surface is the `chip-agent` CLI (`chat` / `run` / `resume`) and a **Textual** TUI; VCD parsing (`pyvcd`) feeds waveform windows into the semantic diagnosis.



Plane / layer	Technology	Role in NoeSI
Control — orchestration	LangGraph `StateGraph` + `langgraph-checkpoint-sqlite`	the seven-stage machine; deterministic gate edges; checkpoint + human-gate interrupt / resume
Control — model gateway	LiteLLM	one `generate` / `stream` API over all backends; per-task + per-escalation

Plane / layer	Technology	Role in NoeSI
		routing; transparent fallback
Control — local serving	Ollama (Qwen-Coder class)	high-volume candidate generation + syntactic repair
Control — frontier models	any LiteLLM provider (e.g. Opus-class)	planning, semantic diagnosis, `EXHAUSTED`-rung repair
Tool — EDA image	IIC-OSIC-TOOLS (pinned tag + digest)	reproducible toolchain bundle
Tool — flow spine	LibreLane (Python Step/Flow API)	deterministic RTL→GDSII driver; agents steer, don't reimplement
Tool — synthesis / P&R	Yosys, OpenROAD	logic synthesis; floorplan → place → CTS → route
Tool — signoff	OpenSTA, Magic, KLayout, Netgen	STA timing, DRC, layout/GDS, LVS
Tool — RTL verification	Verible, Verilator, iverilog, cocotb	lint; compile-elaborate; simulation
Sandbox	Docker	`--network=none`, cpu/mem/time caps, single design-dir mount
PDK	gf180mcuD / sky130A	configurable target process
Data — artifact store	SQLite index + content-addressed blob dir, Pydantic v2	immutable, versioned, typed design state (the blackboard)
Observability — audit	SQLite HMAC-SHA256 hash-chain	tamper-evident, replayable decision log
Observability — trace	JSONL / Langfuse	per-call tokens / latency / cost / escalation
Operator UX	Textual TUI + argparse CLI	`chat` / `run` / `resume`; four-pane TUI
Language / quality	Python 3.12, Pydantic v2, ruff, mypy `--strict`,	typed, lint- and type-clean codebase

Plane / layer	Technology	Role in NoeSI
	pytest	

Agents

The topology is deliberately **one orchestrator + a few stage specialists**, not a swarm of fine-grained agents. Each specialist is a ReAct agent with a single typed responsibility and a task-bound model (`chip_agent/agents/`); the orchestrator (the LangGraph spine) sequences them and enforces the gates. The main agents:

- **SpecIntakeAgent** (`spec_intake.py`, `SPEC_INTAKE`) — normalises an NL spec into a typed `Spec`, asking one clarifying question at a time when a critical field is missing.
- **PlannerAgent** (`planner.py`, `PLAN`) — turns the `Spec` into a `DesignPlan` (module hierarchy, ports, dependencies) as strict JSON.
- **RTLGenerationAgent** (`rtl_gen.py`, `RTL_GEN` / `RTL_REPAIR`) — emits one module's synthesisable Verilog and drives the *inner* loop repair from lint/elaborate violations; the *outer* semantic-repair prompt (`rtl_stage.py`) repairs from a typed `FailureDiagnosis`.
- **Test-first (M19) triangulation agents** — `ContractExtractionAgent` (behavioural contract from the spec), `OracleGenAgent` (an executable Python reference model), `AssertionGenAgent` (three-stage SVA), and the `TestbenchGen` / `differential_tb` / `vector_tb` builders that produce the cocotb gates — cross-checked by `oracle_verification`.
- **Routing agents** — `ReflectionRoutingAgent` (picks one of four recovery routes when the RTL outer loop exhausts) and `PhysicalRepairRoutingAgent` (SIGNOFF timing-failure recovery).
- **Backend stage drivers** — `synth_stage` / `physical_stage` / `signoff_stage` / `gdsii_stage` wrap the real EDA tools behind the gates.
- **HumanHintDistillAgent** (`human_hint_distill.py`) — distils an operator's free-text guidance into a typed `HumanHint` that re-seeds a bounded retry, while being explicitly forbidden from certifying the design (the §4.4 non-authority property, enforced in the prompt itself).

Sample prompts (verbatim excerpts):

```
# SpecIntakeAgent — normalise or ask
You normalise natural-language chip-module specs into a structured
description.
If the input is missing any critical detail (clock domain, reset
polarity,
```

```
port names + widths, or top-level constraints …), respond with a
single-line
JSON object instead: {"type": "question", "question": "…",
"missing_field": "…"}

# RTLGenerationAgent — generate
You are a synthesisable Verilog/SystemVerilog generator. … Output ONLY
the RTL.
* Honour the port list exactly (name, direction, width).
* No latches: every always_ff must register a value on every path …
* No `initial` blocks for synthesisable logic …

# RTL outer semantic-repair — reason over the diagnosis, not raw
stdout
You are a Verilog/SystemVerilog semantic-repair tool. The previous RTL
passes
lint and elaborate but FAILS simulation … Use the typed failure
diagnosis
(not raw simulator output) to find the root cause and emit a corrected
module.

# HumanHintDistillAgent — shape the retry, never certify
Distil their message into ONE structured hint the repair loop can act
on. …
* NEVER claim the design is correct or that the gate should pass. You
are
  only shaping the next repair attempt.
```

How it works

A natural-language spec — typed into the chat REPL or read from a `.md` file — is normalised by the SPEC agent into a typed `Spec`, which the PLAN agent turns into a `DesignPlan` of module declarations. Each stage (RTL, SYNTH, PHYSICAL, SIGNOFF) is a ReAct specialist wrapped in the two-loop repair: it generates or tunes an artifact, runs the stage's real checkers, and on failure repairs within a bounded budget, escalating `INNER → OUTER → EXHAUSTED → HUMAN`. A stage promotes its output to `head` only when the gate reads `gate_ok` from a real verification artifact; the LangGraph edges are pure functions of the blackboard, so the traversal is deterministic and replayable. At SIGNOFF the run pauses at the human gate, and on approval the GDSII stage streams out a real GDS via Magic inside the container. Every transition, gate decision, and model or tool call is appended to the signed audit log and mirrored to named exports the operator can hand-inspect.

---

4. Security implementation

4.1 Gates read real tool oracles, never a self-assessment

A stage transition is a pure function of verification artifacts produced by real tools, not of any text the model emits. The clearest instance is the SIGNOFF gate, which is a **conjunction** over four independent real-tool reports (`chip_agent/agents/signoff_stage.py`):

```
passed = timing_gate_ok and drc.gate_ok and lvs.gate_ok and
security.gate_ok
```

- `timing` / `multi_corner_timing` ← real **OpenSTA**, run against LibreLane's harvested post-CTS SDF and sky130/gf180-mapped netlist (M12–M13, `chip_agent/tools/opensta.py`).
- `drc` ← real **Magic** DRC, cross-checked against LibreLane's own DRC report (`chip_agent/tools/magic_drc.py`).
- `lvs` ← real **Netgen** LVS against the *powered* extracted netlist so port shapes match the SPICE (`chip_agent/tools/netgen_lvs.py`, F13.4-B).
- `security` ← the structural backdoor screen of §4.7.

`SignoffReport.failing_codes()` returns exactly which legs closed the gate (`STA` / `DRC` / `LVS` / `SECURITY`), so a failure is attributable to a tool verdict, never to a narrative. There is no code path in which model output sets `gate_ok`; that is the CLAUDE.md invariant *"NEVER advance a stage on a model's self-assessment"* realised structurally — an injection payload has nothing to overwrite, because the gate is not a thing the agent produces.

4.2 Behavioural gates check ground truth, not a model oracle

For designs that ship published test vectors — like a block cipher — the RTL behavioural gate is a **ground-truth vector testbench** (`chip_agent/agents/vector_tb.py`, F-VECTB). `VectorTBBuilder` renders a transaction-level cocotb testbench that, for each published `(input → known output)` pair, loads the input MSB-first, waits (bounded) on the completion port, unloads the result, and asserts equality against the *spec's* known-good value — emitting a `VEC|…|signal=done|expected=1|actual=0` token on mismatch.

This is trustworthy exactly where a model-generated oracle is not: the PRESENT-80 run showed the model's own oracle can be as wrong as the RTL it checks. `tb_for_module` prefers the vector TB when the module ships spec vectors and is serial-load / completion-gated, and falls back to the differential / LLM testbench

otherwise. A round-reduced or off-by-one cipher fails this gate; a self-graded oracle cannot make it pass.

4.3 A green-but-vacuous result is refused, not promoted

The differential testbench computes `completion_port_never_asserted` (`chip_agent/agents/differential_tb.py`): it finds a 1-bit completion output (`done` / `valid` / …) that the oracle never drives high anywhere across the whole *expected* trace, and rides that finding on the testbench's non-content `metadata["vacuous_completion_port"]`.

When the flag is set, the RTL node's `_check_vacuous_pass` (`chip_agent/graph/state_graph.py`) **refuses to advance the green sim**. It:

1. synthesises a typed `FailureDiagnosis` naming the never-asserted port and the suspected cause ("stimulus window too short to observe completion"),
2. emits a `SIM_VACUOUS_PASS` audit event, and
3. opens an interactive-repair turn — blocking (Option A) or an interrupt/resume pause (Option B) — so **even a non-interactive run halts for review rather than taping out broken silicon.**

It returns `None` (advance normally) only when the pass is genuine or no repair turn is available. A pass that proves nothing is treated as a failure — this is the in-flow fix for the exact 2026-08-08 false-green described in §2.

4.4 A human hint is guidance, never authority

The interactive human-repair turn (M23) lets an operator inject guidance when a stage is stuck, but the operator is **not** a trusted grader. `grant_human_retry` (`chip_agent/graph/human_repair.py`) consumes one bounded `human_turns_used`, resets the attempt budget, and sets the stage to `ESCALATED` — and **deliberately never touches `head`, `last_failure`, or any gate state**, so it can never set `PASSED`. A hint can only *arm another gated attempt*.

Two further hardening details: the distilled hint `summary` is what seeds the retry prompt (`hint_prompt_section`), while the raw operator transcript is provenance-only and excluded from the artifact content hash; and a hallucinated or unresolvable steer (e.g. `REVISIT_SIBLING_RTL` with no resolvable sibling) validates down to `ESCALATE_HUMAN` — re-opening the turn rather than acting on a bad route. A misleading hint that reaches the human costs one bounded retry; it cannot certify an output.

4.5 Sandbox isolation below the framework

Every tool ultimately runs through `SandboxRunner` (`chip_agent/tools/sandbox.py`), which builds a fixed, locked-down `docker run` invocation:

```
docker run --rm --network=none --cpus=<cap> --memory=<cap>m
           --name=chip-agent-<uuid> -v <design-dir>:/work[:ro] -w
/work <pinned-image> …
```

The enforced properties:

- **`--network=none`** — no egress; the PDK is baked into the image, so no download path exists for a compromised step to exfiltrate through or pull code from.
- **`--cpus` / `--memory`** — CPU and RAM caps from `SandboxSettings`.
- **`-v <design-dir>:/work`** — the *only* host directory the container can see; nothing outside the design dir is mountable, and services that only read the design pass `:ro`.
- **outer wall-clock timeout** — on expiry the container is killed (`docker kill`) and the result returns `returncode=124`, so a hung or spinning step cannot stall the run.
- **`--rm` + unique name** — every container is single-use and independently killable.

The docker call is funnelled through a `ProcessRunner` Protocol so the isolation contract is unit-testable with a stub and only integration tests touch a real daemon. Because isolation is enforced here, one layer below LangGraph, a compromise at the orchestration layer does not automatically grant tool-layer or network access.

4.6 Typed state boundary

Inter-stage state is carried only as Pydantic artifacts in the content-addressed store — never as freeform text that could smuggle instructions between stages. Artifacts are immutable: a change writes a new version, and the content hash **excludes** lifecycle fields (status / version / provenance / timestamps), so the design state is a verifiable DAG rather than a mutable blob (`chip_agent/design_state.py`, M1; CLAUDE.md invariants).

4.7 Structural backdoor screen at signoff

`parse_netlist_security` (`chip_agent/tools/security_check.py`, F6.4) runs cheap structural checks over the gate-level netlist and feeds the SIGNOFF `security` leg (§4.1):

- **Always-on nets** — `assign <net> = 1'b1` (and `1'h1` / `1'd1`, bus-bit suffixes included) → `SECURITY.ALWAYS_ON_NET` at **error** severity. A net tied unconditionally high is the textbook backdoor primitive.
- **Always-off nets** — the `1'b0` form → `SECURITY.ALWAYS_OFF_NET` at **warning** severity (pulldowns are common and legitimate — advisory only).
- **Suspicious identifiers** — word-boundaried names matching `backdoor` / `debug_force` / `test_force` / `scan_bypass` / `security_bypass` → `SECURITY.SUSPICIOUS_NAME` at **error**. (`feedback` does not match; `backdoor_disable_n` does.)

The report `passed` iff zero error-severity violations, so any always-on net or suspicious identifier closes the SIGNOFF gate. The parser is pure and unit-tested against a backdoor-net fixture.

4.8 Tamper-evident, signed provenance

Every stage transition and gate decision is recorded in an append-only, hash-chained, HMAC-signed audit log on SQLite (`chip_agent/obs/audit_log.py`, F7.2). Each row carries:

- `content_hash = sha256(` canonical JSON of the event, **excluding** its own hash and signature `)` — so tampering with any field (payload, type, timestamp, sequence) changes the recomputed hash;
- `prev_hash =` the previous event's `content_hash` (or `GENESIS` at sequence 1) — so an inserted, dropped, or reordered event breaks the chain;
- `signature = HMAC-SHA256(key, content_hash)` — so without the signing key an attacker cannot repair a chain they altered.

`verify(design_id)` walks the chain and runs all three checks per event (`hmac.compare_digest` for the signature), returning the first bad sequence. Per-`design_id` chains are independent. The event vocabulary is deliberately small and typed — including the security-relevant `SIM_VACUOUS_PASS`, `HUMAN_TURN_OPENED` / `HUMAN_HINT_RECORDED` / `HUMAN_RETRY_DISPATCHED`, and `ARTIFACT_PROMOTED` — so a run's full decision sequence replays in order and any post-hoc alteration is provable.

4.9 Reproducibility as integrity

Provenance pins the IIC-OSIC-TOOLS image digest, the model + seed + temperature, and the config hash on every artifact; `SandboxRunner.image_digest` surfaces the pinned digest to the `ToolVersion` each service stamps. A deterministic run replays to identical content hashes (F7.3), which is what makes "who changed what, and can we prove it" answerable for an agentic flow.

---

5. Operator UX (human-on-the-loop)

The flow is human-*on*-the-loop by design: mandatory approval at the SIGNOFF human gate before the irreversible GDSII step, plus the interactive "unstuck" turn of §4.4. The operator drives it through a `chip-agent chat` REPL (M11) or a single Textual TUI with chat / pipeline-progress / audit / exports panes (M14). None of this UX can set `gate_ok`; it only surfaces the gated state and, at the human gate, gathers guidance for a bounded retry.

---

6. Demonstration and defense case studies

Vehicle: PRESENT-80 on gf180mcuD. A small, well-specified block cipher whose published test vectors make manipulation *observable*: a round-reduced or key-leaking implementation fails the vectors, and a structurally-correct-but-Trojaned one is the hardest case for the gate stack. `specs/present80.md` ships with the repo; configs target `gf180mcuD` (`configs/frontier-only.yaml`, `configs/local-only.yaml`).

Physical result. The flow drives PRESENT-80 end-to-end to a real gf180mcuD **GDSII stream-out** — DRC-clean and LVS-clean, timing-closed at the nominal (tt) corner under real OpenSTA / Magic DRC / Netgen LVS (§4.1), emitted only after the human gate. The end-to-end run produces a valid multi-megabyte GDS — e.g. `runs/present80_e2e/exports/…/gds/present80_top.gds` (~1.1 MB, opening with the GDSII HEADER record `00 06 00 02`) — not a placeholder. The measured signoff numbers, including the remaining slow-corner timing gap, are in §8.4.

Three reproducible defense case studies, each an existence proof that a defect class is caught by construction:

Defect class	Injected condition	Where it is caught
Backdoor insertion	always-on net / `backdoor`-named signal in the netlist	SIGNOFF structural screen `SECURITY.ALWAYS_ON_NET` / `SECURITY.SUSPICIOUS_NAME` (§4.7)
Cipher weakening	round-reduced / off-by-one PRESENT-80 RTL	ground-truth vector TB fails `VEC|…expected≠actual` (§4.2) — the real 2026-08-08 catch
Verification bypass / false green	a `done`-gated design whose completion never asserts	vacuous-pass refusal `SIM_VACUOUS_PASS`, run halts for review (§4.3)

7. Deliverables and repository pointers

Deliverable / claim	Code
Verification-gated seven-stage spine + two-loop repair	`chip_agent/graph/state_graph.py`, `docs/control_graph.md`
Real-oracle SIGNOFF conjunction	`chip_agent/agents/signoff_stage.py`, `chip_agent/tools/{opensta,magic_drc,netgen_lvs}.py`
Ground-truth vector testbench	`chip_agent/agents/vector_tb.py`
Vacuous-pass refusal	`chip_agent/agents/differential_tb.py`, `chip_agent/graph/state_graph.py` (`_check_vacuous_pass`)
Non-authoritative human hint	`chip_agent/graph/human_repair.py`
Network-less sandbox	`chip_agent/tools/sandbox.py`
Typed content-addressed state	`chip_agent/design_state.py`, `chip_agent/store/`
Structural backdoor screen	`chip_agent/tools/security_check.py`
Tamper-evident signed audit log	`chip_agent/obs/audit_log.py`
Reproducibility / replay	`chip_agent/obs/replay.py`, pinned

Deliverable / claim	Code
	image digest in `SandboxRunner`
Operator UX (chat + TUI)	`chip_agent/cli.py` (`chat`), `chip_agent/tui/`
PRESENT-80 vehicle	`specs/present80.md`, `configs/frontier-only.yaml`
Backlog / status	`docs/FEATURES.md` (M0–M24), `docs/KNOWN_ISSUES.md`

8. Chipathon deliverables, evidence & roadmap

This section maps the tapeout-track deliverables to their current status. Where a number exists it is the measured value from a committed run; where work remains it is called out plainly.

8.1 Block freeze (PRESENT-80)

Parameter	Value
Block	PRESENT-80, encryption-only, iterative (one round/clock, 31 rounds), ISO/IEC 29192-2
Top module	`present80` — a single flat, Yosys-synthesizable module
I/O pins	**7** (`clk`, `rst_n`, `load_en`, `din`, `shift_out_en`, `dout`, `done`) — bit-serial to fit the pin budget (a parallel interface would need ~212)
Block / key	64-bit block, 80-bit key
Clock target	10 ns / 100 MHz (`target_clock_ns` in `configs/*.yaml`)
PDK / std cells	gf180mcuD / `gf180mcu_fd_sc_mcu7t5v0`
Std-cell instances	3,183 (7,867 incl. fill)
Die / core area	471.4 × 489.3 µm ≈ 0.231 mm² die; 0.208 mm² core
Utilization	26.7%

The block, pin, area, and clock budget are committed to the repo (`specs/present80.md` + config). *[To finalise: co-sign the pin map + area/pin budget with the integration track.]*

8.2 Source and logs in-repo (not behind links)

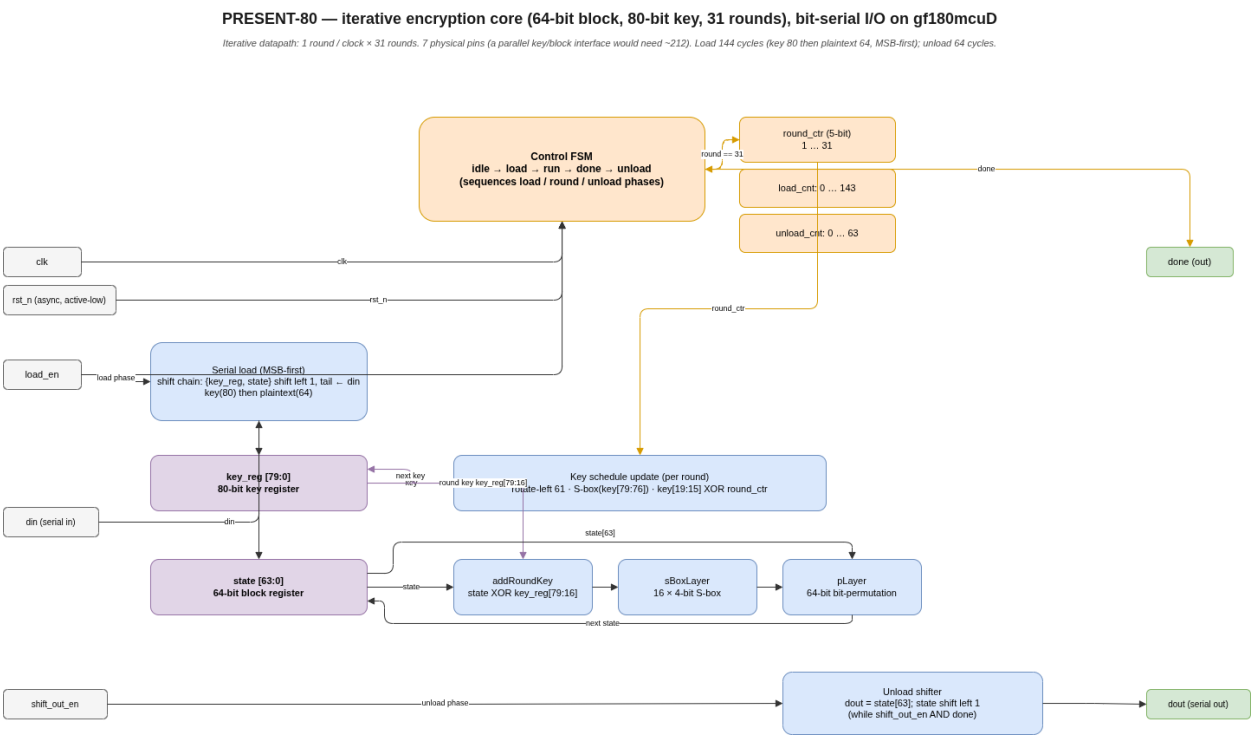
The PRESENT-80 spec is committed (`specs/present80.md`); the generated RTL, testbenches, and signoff logs currently live under the **gitignored** `runs/` store. **Deliverable:** promote the frozen PRESENT-80 RTL, the vector + gate-level testbenches, and the STA / DRC / LVS logs into tracked fixtures (e.g. `tests/fixtures/present80/`) so they are reviewable in-repo rather than via Drive links. The `runs/` gitignore is why the external history looks quiet since mid-June despite active M12–M24 work; committing these artifacts closes that perception gap.

8.3 Functional verification against published vectors

The ground-truth **vector testbench** already exists (`chip_agent/agents/vector_tb.py`, §4.2): it loads each published `(key, plaintext → ciphertext)` vector MSB-first, waits on `done`, unloads, and asserts equality. **Deliverable:** a committed **test list + pass/fail table** over the ISO/IEC 29192-2 vectors, **one waveform** of a full 64-bit encryption (load → 31 rounds → unload), and **gate-level simulation** on the post-synthesis netlist back-annotated with **SDF** (the harvest already ingests the CTS SDF, §4.1).

8.4 GF180MCU signoff numbers (measured; logs to commit)

From a committed PRESENT-80 run (`runs/present80_app_gdsii/metrics.json`); logs to be promoted per §8.2:



Metric	Value
Setup WNS / TNS — tt, 25 °C, 5.0 V	0 ns / 0 ns — **closed at nominal corner**
Setup WNS / TNS — ss, 125 °C, 4.5 V	−2.53 ns / −260 ns — **slow-corner closure gap (open)**
Hold violations (all corners)	0
Magic DRC errors	**0**
Netgen LVS (device / net / pin mismatches)	**0 — LVS clean**
Antenna violations	0
Total power (tt)	≈ 78 mW

Honest status: DRC-clean, LVS-clean, and timing-closed at the typical corner; the slow corner (ss / 125 °C) still carries setup violations. Closing it — buffering, a relaxed clock target, or a multi-corner-aware physical repair (F21.x) — is a remaining task.

8.5 Security claims — stated precisely

- **What the SIGNOFF security gate checks :** a structural netlist screen (`security_check.py`, §4.7) flagging unconditional always-on nets

(`SECURITY.ALWAYS_ON_NET`) and suspicious identifiers (`backdoor` / `scan_bypass` / …, `SECURITY.SUSPICIOUS_NAME`), plus the gate-honesty properties of §4 (real-oracle gates, ground-truth vectors, vacuous-pass refusal). This is a real and *structural* check.

8.6 Bring-up plan (the program ends in measurement this year)

The 7-pin bit-serial interface **is** the test interface — no extra DFT harness is needed to exercise the block on the packaged part:

1. Hold `rst_n` low; release synchronous to `clk`.
2. Raise `load_en` and clock in 80 key bits then 64 plaintext bits, MSB-first, on `din` (144 cycles).
3. Wait for `done` (≈ 31 round cycles).
4. Raise `shift_out_en` and clock out 64 ciphertext bits, MSB-first, on `dout`.
5. Compare against the ISO/IEC 29192-2 vector; sweep `clk` to find the maximum frequency and cross-check against STA.

[To finalise: packaged pin assignment + measurement board / harness with the integration + measurement track.]

8.7 Roadmap

Milestone	Target	Owner	Status
Gate 0 static config scanner (§9)	*September 2026*	*Eleftherios Batzolis*	Designed, testing for integration
Red-team corpus + hardened-vs-unhardened ASR/UAR study (§9)	*September 2026*	*Eleftherios Batzolis*	future
Packaged bring-up + measurement (§8.6)	*When delivered*	*Eleftherios Batzolis* *Konstantinos Rantos* *Drosatos Georgios*	plan drafted

Risks. Slow-corner timing may force an area/utilization or clock relaxation; closed-source API nondeterminism (mitigated by frozen invocation capture).

9. Future work — designed but not yet implemented

The mechanisms above make the following tractable; none is claimed as built.

- **Gate 0 — static pre-flight config audit.** A deterministic, zero-LLM-call scanner over the agent configuration itself (secrets / credentials, wildcard tool-permission scopes, hook and shell-injection risk, MCP-server risks, hidden-Unicode / base64 / auto-run directives), released as a standalone open-source package and run before any spec is processed. This is the one v5 defense layer not yet built; it slots in ahead of SPEC.
- **Empirical adversarial-robustness study.** A frozen red-team corpus (30–50 adversarial specs across Trojan-insertion / cipher-weakening / verification-bypass), an *unhardened* baseline arm, a per-defense-layer ablation harness, and pre-registered statistics (Attack Success Rate, Unsafe Action Rate, Tapeout Survival, stage-of-detection kill-chain, McNemar's test, exact binomial CIs). This turns §7's qualitative case studies into a measured hardened-vs-unhardened result.
- **Cross-model / cross-family robustness panel.** Run the evaluation across a panel of open- and closed-source models to rule out single-model RLHF artifacts. The router already addresses multiple backends; the cross-model *security* evaluation is not built.
- **Secondary Attacker/Defender/Auditor pipeline.** A three-subagent generative adversarial pass producing attacks not constrained to the frozen corpus.
- **Formal equivalence (stretch).** SymbiYosys bounded model checking establishing functional equivalence between the produced RTL and a hand-written reference within a bounded depth, catching Trojans whose trigger lies inside that depth.